



Recibe una cálida:

# ¡Bienvenida!

---

Te estábamos esperando 😊 +

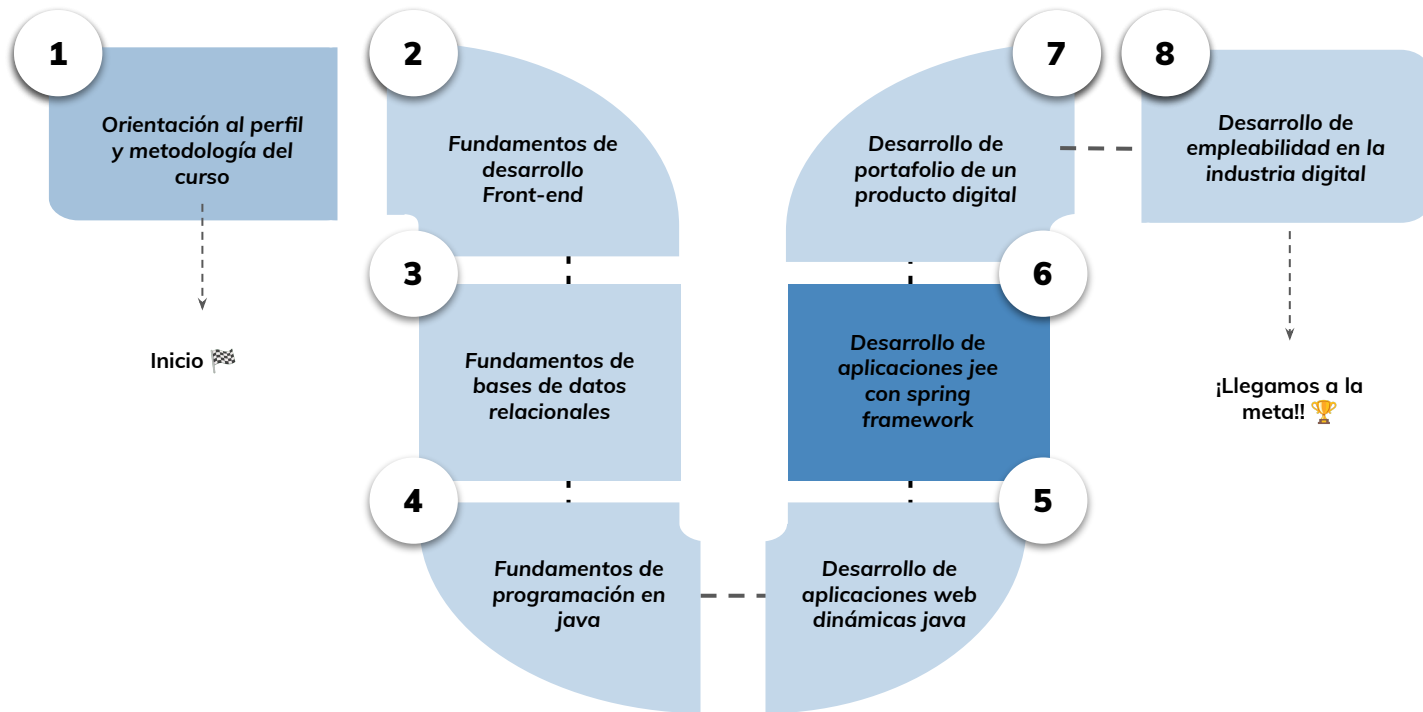


# › Control de acceso mediante spring security - Parte II

**Aprendizaje Esperado 4:** Implementar mecanismos de seguridad utilizando spring security para controlar el acceso a los recursos del aplicativo.

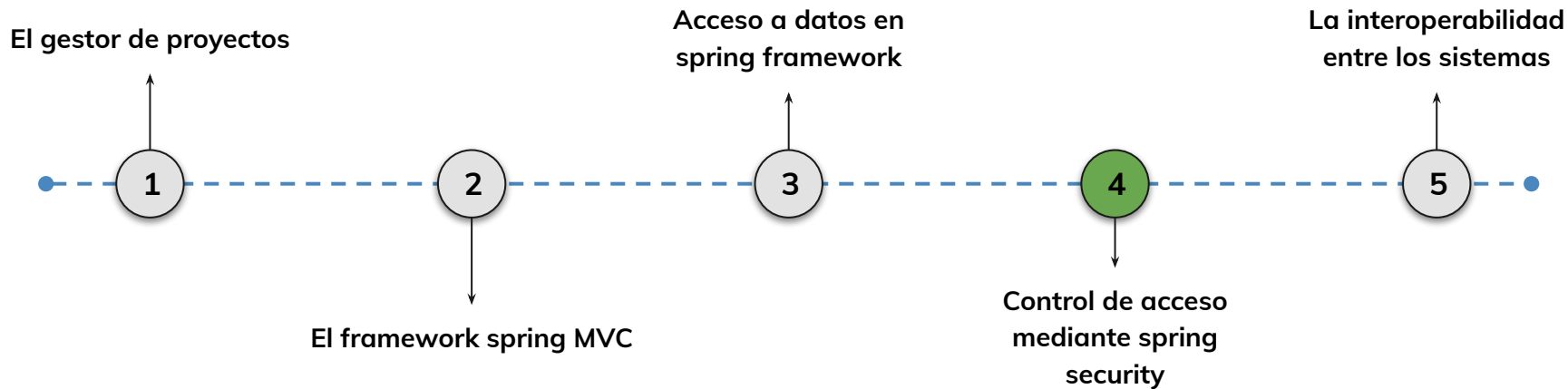
# Hoja de ruta

¿Cuáles skills conforman el programa? **Desarrollo de Aplicaciones Full Stack Java Trainee**



# Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



# Learning Path

¿Cuáles temas trabajaremos hoy?

4.

Control de seguridad  
visual y en base de datos

Implementar seguridad avanzada en aplicaciones Spring: acceso condicional en la vista (JSP o Thymeleaf), recuperación del usuario autenticado en el backend y almacenamiento seguro de contraseñas con JPA.

Seguridad en la vista

Uso de taglibs y atributos access

Visualización condicional según roles y autenticación

Seguridad y autenticación  
con JPA

Encriptación de contraseñas con  
BCryptPasswordEncoder

Personalización con  
AuthenticationManagerBuilder



# Objetivos de aprendizaje

¿Qué aprenderás?



- Filtrar contenido en la vista según el rol del usuario.
- Obtener datos del usuario autenticado en el controlador.
- Integrar Spring Security con JPA para autenticación.
- Utilizar BCryptPasswordEncoder para guardar contraseñas seguras.
- Crear un formulario de registro funcional que codifique contraseñas al persistir.

# Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Incorporación de Spring Security al proyecto
- Configuración de rutas públicas y protegidas
- Implementación de la interfaz UserDetailsService
- Creación de un formulario de login personalizado

# Control de acceso mediante spring security



# Añadiendo seguridad a los elementos de la vista

Spring Security cuenta con **taglib** que nos provee acceso a la información de seguridad de los usuarios y posibilita el filtrado del contenido que se muestra al usuario en base a sus privilegios. Para utilizar **taglib** en una página **JSP**, debemos agregar la siguiente dependencia:

```
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-taglibs</artifactId>
  <version>${security.version}</version>
</dependency>
```

# Añadiendo seguridad a los elementos de la vista

Y declaramos el taglib en el archivo JSP de la siguiente manera:

```
*index.jsp X
1 <%@ taglib prefix="sec" uri="http://www.springframework.org/security/tags" %>
2
```

# Añadiendo seguridad a los elementos de la vista

Si en lugar de utilizar una página JSP, decidimos utilizar Thymeleaf con HTML, debemos incluir la dependencia:

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-thymeleaf</artifactId>  
</dependency>
```

# Añadiendo seguridad a los elementos de la vista

Y agregar la declaración de seguridad en el HTML, de ésta manera:

```
<!DOCTYPE html>  
<html xmlns:th="http://www.thymeleaf.org"  
      xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
```

# Añadiendo seguridad a los elementos de la vista

Una vez configurado el prefijo “**sec**”, podemos acceder a etiquetas que nos permitirán acceder a diversas personalizaciones.

La etiqueta **authorize** nos sirve para **filtrar el contenido que se mostrará en base a los privilegios del usuario**, por ejemplo, podemos definir un contenido que solo será mostrado a aquellos usuarios con privilegios de administrador, esta etiqueta la utilizaremos de la siguiente manera:

```
<sec:authorize access="hasRole('ROLE_ADMIN')">

    Este contenido se muestra si el usuario tiene el privilegio:
    <b>ROLE_ADMIN</b>.

</sec:authorize>
```

# Añadiendo seguridad a los elementos de la vista

Estableciendo el atributo `access="isAuthenticated()"` **mostraremos contenido a todos los usuarios autenticados** .

```
<sec:authorize access= access="isAuthenticated() and principal.username == 'jesus'">
    Este contenido se muestra si:
    <b>Si el username del usuario autenticado es "jesus"</b>.
</sec:authorize>
```

# Añadiendo seguridad a los elementos de la vista

Otra forma de usar esta etiqueta es indicando el atributo `url`, por medio del mismo, el contenido será generado si el usuario actual tiene permiso para acceder a la URL indicada.

```
<sec:authorize url="/user/delete">

    Este contenido se muestra si:
    <b>Si el usuario autenticado tiene permisos para acceder a
    la url especificada</b>.

</sec:authorize>
```

# Añadiendo seguridad a los elementos de la vista

Si deseamos obtener el nombre del usuario y sus roles, usaremos las siguientes etiquetas:

```
Nombre de usuario:  
<sec:authentication property="principal.username" />  
  
Roles de usuario.  
<sec:authentication property="principal.authorities" />
```



➤ Obteniendo el usuario  
autenticado en el controlador



# Obteniendo el usuario autenticado en el controlador

Para obtener el usuario autenticado en un controlador en una aplicación Spring Security, podemos hacer uso de la clase `SecurityContextHolder`. Esta clase proporciona acceso al contexto de seguridad de Spring, y puedes extraer información sobre el usuario autenticado. El usuario autenticado generalmente se almacena en un objeto `Authentication`.



# Obteniendo el usuario autenticado en el controlador

Primero debemos tener los imports necesarios:

- `import org.springframework.security.core.Authentication;`
- `import org.springframework.security.core.context.SecurityContextHolder;`
- `import org.springframework.security.core.userdetails.UserDetails;`

# Obteniendo el usuario autenticado en el controlador

Luego, en el método del controlador donde deseamos obtener el usuario actual, podemos hacer lo siguiente:

```
@GetMapping("/profile")
public String userProfile(Model model) {
    // Obtener el contexto de seguridad actual
    Authentication authentication = SecurityContextHolder.getContext().getAuthentication();

    // Obtener el nombre de usuario del usuario autenticado
    String username = authentication.getName();

    // Puedes obtener más información sobre el usuario, como roles, detalles, etc.
    // UserDetails userDetails = (UserDetails) authentication.getPrincipal();
    // List<GrantedAuthority> authorities = (List<GrantedAuthority>) authentication.getAuthorities();

    // Agregar el nombre de usuario al modelo para mostrarlo en la vista
    model.addAttribute("username", username);

    // Retornar la vista del perfil del usuario
    return "profile";
}
```

# Obteniendo el usuario autenticado en el controlador

- `SecurityContextHolder.getContext().getAuthentication()`: Obtiene el contexto de seguridad actual y, a partir de él, obtiene la información de autenticación.

```
@GetMapping("/profile")
public String userProfile(Model model) {
    // Obtener el contexto de seguridad actual
    Authentication authentication = SecurityContextHolder.getContext().getAuthentication();

    // Obtener el nombre de usuario del usuario autenticado
    String username = authentication.getName();

    // Puedes obtener más información sobre el usuario, como roles, detalles, etc.
    // UserDetails userDetails = (UserDetails) authentication.getPrincipal();
    // List<GrantedAuthority> authorities = (List<GrantedAuthority>) authentication.getAuthorities();

    // Agregar el nombre de usuario al modelo para mostrarlo en la vista
    model.addAttribute("username", username);

    // Retornar la vista del perfil del usuario
    return "profile";
}
```



# Obteniendo el usuario autenticado en el controlador

`authentication.getName()`: Extrae el nombre de usuario del objeto `Authentication`. También puedes acceder a otras propiedades del usuario autenticado según tu configuración, como roles, detalles, etc.

```
@GetMapping("/profile")
public String userProfile(Model model) {
    // Obtener el contexto de seguridad actual
    Authentication authentication = SecurityContextHolder.getContext().getAuthentication();

    // Obtener el nombre de usuario del usuario autenticado
    String username = authentication.getName();

    // Puedes obtener más información sobre el usuario, como roles, detalles, etc.
    // UserDetails userDetails = (UserDetails) authentication.getPrincipal();
    // List<GrantedAuthority> authorities = (List<GrantedAuthority>) authentication.getAuthorities();

    // Agregar el nombre de usuario al modelo para mostrarlo en la vista
    model.addAttribute("username", username);

    // Retornar la vista del perfil del usuario
    return "profile";
}
```



# Obteniendo el usuario autenticado en el controlador

`model.addAttribute("username", username)`: Agrega el nombre de usuario al modelo de Spring, lo que permitirá acceder a esta información en la vista asociada.

`return "profile"`: Retorna el nombre de la vista que mostrará el perfil del usuario.

```
@GetMapping("/profile")
public String userProfile(Model model) {
    // Obtener el contexto de seguridad actual
    Authentication authentication = SecurityContextHolder.getContext().getAuthentication();

    // Obtener el nombre de usuario del usuario autenticado
    String username = authentication.getName();

    // Puedes obtener más información sobre el usuario, como roles, detalles, etc.
    // UserDetails userDetails = (UserDetails) authentication.getPrincipal();
    // List<GrantedAuthority> authorities = (List<GrantedAuthority>) authentication.getAuthorities();

    // Agregar el nombre de usuario al modelo para mostrarlo en la vista
    model.addAttribute("username", username);

    // Retornar la vista del perfil del usuario
    return "profile";
}
```

# Live Coding

## ¿En qué consistirá la Demo?

Vamos a recuperar el nombre de usuario actualmente autenticado en el backend (controlador), utilizando Spring Security y mostrarlo en la vista. Este dato se podrá usar para personalizar pantallas, registros, menús o perfiles de usuario.

1. Importar las clases necesarias  
Authentication, SecurityContextHolder y UserDetails.
2. Obtener el usuario autenticado  
Usamos  
`SecurityContextHolder.getContext().getAuthentication()`.
3. Extraer nombre de usuario  
Con `authentication.getName()` obtenemos el nombre.
4. Enviar el dato a la vista  
Usamos `model.addAttribute("username", username)`.
5. Crear controlador /perfil  
Para mapear y retornar la vista `perfil.html`.
6. Diseñar una vista con Thymeleaf  
Mostramos el nombre usando  
`th:text="${username}"`.
7. Configurar seguridad en SecurityConfig  
Aseguramos que /perfil esté protegido por autenticación.



## ➤ Autenticación contra una base de datos utilizando JPA

# < > Autenticación utilizando JPA

La autenticación es un componente esencial en cualquier aplicación web para garantizar que solo usuarios autorizados tengan acceso a los recursos protegidos. Spring Security proporciona un sólido marco para implementar la autenticación, y cuando se combina con Java Persistence API (JPA), la autenticación contra una base de datos se vuelve más potente y flexible.



# < > Autenticación utilizando JPA

## Rol de JPA en la Autenticación:

Java Persistence API (JPA) es una especificación de Java para el mapeo objeto-relacional, lo que facilita el almacenamiento y la recuperación de objetos Java en bases de datos relacionales. Al utilizar JPA en combinación con Spring Security, puedes aprovechar su capacidad para interactuar directamente con la capa de persistencia de la base de datos para autenticar usuarios.



# ➤ Autenticación y encriptación de contraseñas mediante JPA

# < > Autenticación y encriptación de contraseñas mediante JPA

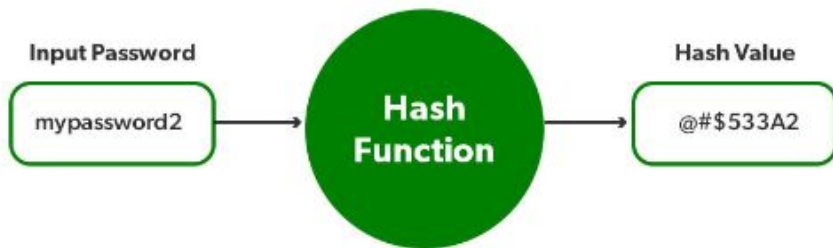
Un mecanismo de seguridad que hay que tener en cuenta a la hora de guardar los datos del usuario es **proteger las contraseñas**.

Usualmente no se almacena la contraseña como tal, sino que lo que guardamos en la base de datos es el **valor hash** de las mismas. De este modo, si alguien logra tener acceso a nuestra base de datos de usuarios, no podrá leer las contraseñas.



# < > Autenticación y encriptación de contraseñas mediante JPA

Para esta tarea Spring Security utiliza la interfaz `PasswordEncoder`, podemos crear nuestra propia implementación o utilizar una de las proporcionadas por el Framework, para este ejemplo utilizaremos la clase `BCryptPasswordEncoder`.



# < > Autenticación y encriptación de contraseñas mediante JPA

En el método `configureGlobal` de la clase de seguridad, tendremos el objeto `AuthenticationManagerBuilder` que se encarga de realizar el manejo de la autenticación de usuarios.

A continuación del llamado a `userDetailsService` para autenticar, colocaremos otro punto para llamar al `passwordEncoder`.

```
@Autowired
public void configureGlobal(AuthenticationManagerBuilder auth) throws Exception{
    auth.userDetailsService(usuarioServicio)
        .passwordEncoder(new BCryptPasswordEncoder());
}
```



# < > Autenticación y encriptación de contraseñas mediante JPA

Una vez definido el método de autenticación y encriptación de contraseñas, solo nos resta **setear el encoder al momento de guardar la contraseña** generada por el usuario.

**Es necesario que en la base de datos quede persistida la contraseña ya codificada** . Esto lo realiza JPA con referencia de las anotaciones ya mencionadas.





# < > Autenticación y encriptación de contraseñas mediante JPA

Para poder definir la encriptación al momento de la creación de la contraseña, vamos a agregar el llamado al encoder en el método de creación de usuario en el servicio. Por ejemplo:

```
UserEntity user = new UserEntity();  
  
user.setName(name);  
user.setEmail(email);  
  
user.setPassword(new BCryptPasswordEncoder().encode(password));  
  
user.setRole(Role.USER);  
  
userRepository.save(user);
```



# < > Autenticación y encriptación de contraseñas mediante JPA

De esta manera estamos pidiéndole a JPA que **codifique la contraseña en el mismo momento en el que se setee en la variable password** . Si realizamos una consulta en la base de datos, podremos ver que la contraseña está compuesta de caracteres alfanuméricos en cadena larga.

```
UserEntity user = new UserEntity();

user.setName(name);
user.setEmail(email);

user.setPassword(new BCryptPasswordEncoder().encode(password));

user.setRole(Role.USER);

userRepository.save(user);
```



# < > Autenticación y encriptación de contraseñas mediante JPA

Por ejemplo, si al momento de crear el usuario colocamos como contraseña la cadena “123456”, la clase `BCryptPasswordEncoder`, generará una contraseña como ésta:

```
$2a$10$fYoIYBQcszFwBxwT3hbjb.QY6TDz1SvDDi9SQgY028maC6khfg2di
```

Y ese será el **valor hash** con el que queda guardada en la base de datos.



# < > Autenticación y encriptación de contraseñas mediante JPA

**Características** de la autenticación con JPA:

1. **Integración Sencilla:** La integración de Spring Security con JPA es fluida y simplifica la implementación de la autenticación, ya que permite utilizar entidades JPA para representar a los usuarios y roles.
2. **Seguridad y Flexibilidad** : Utilizar JPA para la autenticación permite una gestión segura y flexible de usuarios y roles en la base de datos, lo que facilita la adaptación a cambios en los requisitos de seguridad.
3. **Contraseña Segura** : Almacenar contraseñas de manera segura es fundamental. Spring Security facilita esto mediante el uso de codificadores de contraseñas como **BCrypt**, que son difíciles de descifrar.



# < > Autenticación y encriptación de contraseñas mediante JPA

**Características** de la autenticación con JPA:

- 4. **Personalización de Detalles del Usuario** : La interfaz **UserDetails** permite personalizar los detalles del usuario, como roles y autorizaciones, según los requisitos de la aplicación.
- 5. **Manejo de Sesiones** : Spring Security maneja automáticamente la creación y administración de sesiones de usuario, proporcionando una capa adicional de seguridad.



# < > Autenticación y encriptación de contraseñas mediante JPA

Algunas **desventajas** de la autenticación con JPA:

1. **Complejidad en Implementaciones Avanzadas** : A medida que las necesidades de autenticación se vuelven más complejas, como la implementación de flujos de autenticación personalizados, la configuración puede volverse más complicada.
2. **Posible Sobrecarga de Base de Datos** : Dependiendo del tamaño de la aplicación y la cantidad de usuarios, el acceso frecuente a la base de datos para autenticar usuarios podría generar cierta sobrecarga.
3. **Seguridad en la Configuración** : Asegúrate de configurar adecuadamente Spring Security para proteger las configuraciones sensibles, como las credenciales de la base de datos y las claves de codificación de contraseñas.



# < > Autenticación y encriptación de contraseñas mediante JPA

## Consideraciones adicionales :

- **Auditoría y Registros** : Considera la implementación de registros de auditoría para realizar un seguimiento de eventos de autenticación, incluyendo intentos fallidos y exitosos.
- **Actualizaciones de Contraseña** : Implementa una funcionalidad para que los usuarios puedan actualizar sus contraseñas y garantiza que las contraseñas antiguas sean almacenadas de manera segura.
- **Mantenimiento de Usuarios** : Incluye funcionalidades para administrar usuarios, como la capacidad de desactivar cuentas o restablecer contraseñas.



# LIVE CODING

## Encriptando contraseñas

Vamos a agregar las configuraciones de encriptación de contraseñas en la clase de seguridad. Además, vamos a encriptar la contraseña en el momento en que seteamos el atributo.

1. Personalizar el método **configureGlobal** para agregar el encriptador
2. Agregar el encriptador al seteo de la contraseña, para que se codifique al momento de la creación del usuario.
3. Crear un formulario de Registro de Usuario (HTML o JSP)
4. Crear un controlador de registro que reciba los datos del formulario y guarde un usuario creado con su contraseña codificada
5. consultar la base de datos para confirmar que la contraseña fue encriptada con éxito.

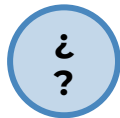
**Tiempo:** 30 minutos



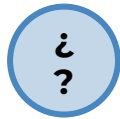
# #Momentode Preguntas...



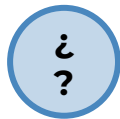
¿Qué diferencia hay entre usar `access="isAuthenticated()"` y `access="hasRole('ADMIN)'"`?



¿Qué método se usa para codificar una contraseña antes de guardarla?



¿Cómo se accede al nombre del usuario autenticado en un controlador?



¿Qué riesgos existen si no se encripta la contraseña en la base de datos?



Momento:

# Time-out!



5 -10 min.





Ejercicio N° 1

# Registro seguro en la Biblioteca Digital

# Registro seguro en la Biblioteca Digital

## Contexto: 🙌

Tu equipo ha finalizado el login seguro en la plataforma de biblioteca digital, pero ahora necesita permitir que nuevos usuarios se registren. Para ello, deberás desarrollar un formulario de registro que almacene la información en la base de datos y asegure la contraseña mediante encriptación.

## Consigna: 📝

Implementa un formulario de registro de usuario utilizando Spring Security + JPA, asegurando que las contraseñas se almacenen en forma encriptada. Crea el controlador correspondiente, configura el codificador de contraseñas en la clase de seguridad y asegúrate de que los usuarios puedan autenticarse luego del registro.

Tiempo 🕒: 30 minutos

# Registro seguro en la Biblioteca Digital

## Paso a paso:

1. Agrega la dependencia BCryptPasswordEncoder en tu proyecto.
2. Define un @Bean de PasswordEncoder en SecurityConfig.
3. Crea un formulario HTML/JSP con campos para nombre, email y contraseña.
4. En el servicio de usuario, aplica encoder.encode() al guardar la contraseña.
5. Configura el AuthenticationManagerBuilder para usar ese encoder.
6. Confirma que la contraseña se guarde en la base de datos codificada.
7. Usa etiquetas sec:authorize en las vistas para mostrar contenido según el rol.

¿Alguna consulta?





# Resumen

¿Qué logramos en esta clase?



- ✓ **Aplicar seguridad en elementos visuales según roles.**
- ✓ **Recuperar la sesión del usuario autenticado desde el controlador.**
- ✓ **Implementar encriptación de contraseñas con BCrypt.**
- ✓ **Integrar JPA con Spring Security para registrar y autenticar usuarios.**
- ✓ **Usar taglibs (JSP) o sec:authorize (Thymeleaf) para contenido condicional.**
- ✓ **Validar el funcionamiento de seguridad en vistas y formularios.**



## ¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.



< **¡Muchas gracias!** >

